Depth-First Search

Deep exploration before backtracking (DFS)

Overview

Depth-First Search (DFS) explores a graph or tree by going as deep as possible along each branch before backtracking. It uses a Last-In-First-Out (LIFO) structure — either an explicit stack or the recursion call stack — to remember which node to return to once a branch is fully explored. DFS is useful when the solution is likely to be deep, or when memory is limited.

Algorithm Steps

1. Push the start node onto a stack (or call it recursively) and mark it visited.
2. Pop (or enter) the current node and process it (e.g., check if it is the goal).
3. Push all unvisited neighbors of that node onto the stack, marking them visited.
4. Continue popping and expanding the most recently added node.
5. Backtrack when a node has no unvisited neighbors; repeat until the stack is empty.

Complexity

Metric	Value
Time Complexity	O(V + E) for a graph with V vertices and E edges
Space Complexity	O(V) worst case, but often O(h) for tree height h with recursion
Completeness	Complete on finite graphs; may loop forever on infinite paths
Optimality	Not optimal — does not guarantee the shortest path

Advantages & Disadvantages

Advantages

✓ Uses less memory than BFS for deep, narrow graphs
✓ Simple recursive implementation
✓ Well suited for tasks like topological sorting and cycle detection

Disadvantages

✗ May get stuck exploring a long, incorrect path
✗ Does not guarantee the shortest or optimal path
✗ Can run into infinite loops on graphs with cycles unless visited nodes are tracked

Typical Use Cases

- Solving mazes and puzzles (e.g., Sudoku, N-Queens) via backtracking
- Topological sorting of tasks or dependencies
- Detecting cycles in a graph

- Exploring file systems and directory trees

C++ Implementation

C++

```cpp
#include <vector>
using namespace std;

void dfs(int node, vector<vector<int>>& adj, vector<bool>& visited) {
    visited[node] = true;
    cout << node << " ";  // process node

    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor, adj, visited);
        }
    }
}

// Call it like this:
// vector<bool> visited(n, false);
// dfs(start, adj, visited);
```